

Gaming Memory Reclaim Prediction System – Deployment Architecture

Overview

The Gaming Memory Reclaim Prediction System is deployed entirely on the Android device and operates without requiring any cloud connectivity. The objective is to predict the amount of memory likely to be reclaimed under the current gaming workload and proactively optimize memory resources before severe memory pressure impacts gameplay.

The deployment consists of telemetry collection, feature preprocessing, ONNX-based inference, memory optimization, and continuous monitoring.

Deployment Artifacts

The Android application package contains the following files:

```
assets/  
|  
├─ gaming_memory_reclaim.onnx  
├─ scaler.json  
├─ iqr_bounds.json  
└─ feature_config.json
```

gaming_memory_reclaim.onnx

Contains the trained XGBoost regression model converted to ONNX format.

scaler.json

Stores StandardScaler parameters used during training.

```
feature_names  
mean  
scale
```

iqr_bounds.json

Stores lower and upper clipping bounds for each feature.

feature_config.json

Stores the expected feature order used during training and inference.

Gaming Session Detection

The system remains idle until a game becomes the foreground application.

```
Game Launch
  ↓
Activate Monitoring Service
```

This minimizes CPU and battery overhead when gaming is not active.

Runtime Telemetry Collection

During active gameplay, a lightweight Android service periodically collects telemetry from system interfaces.

Examples include:

```
Total Memory
Available Memory
Cached Memory
Swap Utilization
DMA Buffer Usage
Heap Usage
DHA Statistics
CPU Load
CPU Temperature
GPU Information
```

Data is obtained from Android system services and Linux kernel interfaces such as:

```
/proc/meminfo
/proc/vmstat
/sys/class/thermal
Samsung memory monitoring interfaces
```

Game Context Collection

The deployment also captures game-specific context.

Examples:

```
Game Package Name
Graphics Setting
Resolution
Refresh Rate
Game Mode
```

This information is included because memory behavior varies significantly across games and graphics configurations.

Feature Processing Pipeline

The collected telemetry snapshot is transformed into the exact feature representation used during model training.

Step 1 — IQR-Based Outlier Handling

Runtime values are clipped using training-time IQR bounds.

```
value < lower_bound
    ↓
replace with lower_bound

value > upper_bound
    ↓
replace with upper_bound
```

This prevents abnormal runtime values from destabilizing predictions.

Step 2 — Feature Preparation

Features are arranged according to the exact order expected by the model.

Example:

```
[
  memtotal_kb,
  memavailable_kb,
  cached_kb,
  swap_utilization_pct,
  dma_buf_kb,
  heapsize_kb,
  currentcpuload,
  aptemp,
  graphics_setting,
```

```
resolution,  
...  
]
```

The feature order must match FEATURE_COLS used during training.

Step 3 — StandardScaler Normalization

Scaling parameters are loaded from:

```
scaler.json
```

Normalization is applied as:

```
scaled_value =  
(value - mean) / scale
```

This guarantees consistency between training and deployment.

ONNX Runtime Initialization

During application startup:

```
Load gaming_memory_reclaim.onnx  
Load scaler.json  
Load iqr_bounds.json  
Create ONNX Runtime Session
```

Initialization occurs only once.

The model remains resident in memory throughout the gaming session.

Inference Pipeline

Each prediction cycle follows:

```
Telemetry Collection  
↓  
Game Context Collection  
↓
```

```
Apply IQR Bounds
↓
Feature Ordering
↓
StandardScaler
↓
ONNX Runtime Inference
↓
Predicted Reclaim Memory (KB)
```

Example output:

```
Predicted Reclaim = 420 MB
```

The prediction represents the estimated amount of memory likely to be reclaimed under the current device state.

Memory Optimization Engine

The predicted reclaim amount is converted into memory-management actions.

Low Memory Pressure

```
Predicted Reclaim < 100 MB
```

Action:

```
No intervention required
```

Moderate Memory Pressure

```
Predicted Reclaim 100-300 MB
```

Action:

```
Light cache cleanup
```

High Memory Pressure

Predicted Reclaim 300-600 MB

Action:

Aggressive cache cleanup
Background memory optimization

Critical Memory Pressure

Predicted Reclaim > 600 MB

Action:

Maximum memory optimization
Game Booster integration

The objective is to reduce sudden reclaim events that may introduce frame drops, lag, or gameplay stutter.

Logging and Monitoring

Every prediction is logged locally.

Stored information:

```
timestamp  
game_name  
graphics_setting  
predicted_reclaim_kb  
actual_reclaim_kb  
error_kb  
action_taken
```

These logs are used for:

- Monitoring model accuracy
- Drift detection
- Future retraining
- Deployment validation

OTA Model Updates

When a new model version becomes available:

```
Training Team Retrains Model
      ↓
Generate New ONNX Model
      ↓
OTA Deployment
      ↓
Device Downloads Artifacts
      ↓
Background Service Loads New Model
      ↓
Old Model Removed
```

Updated artifacts include:

```
gaming_memory_reclaim.onnx
scaler.json
iqr_bounds.json
```

No user interaction is required.

Performance Characteristics

Typical runtime costs:

Kernel Data Collection	~5 ms
Feature Processing	~2 ms
ONNX Inference	~1 ms

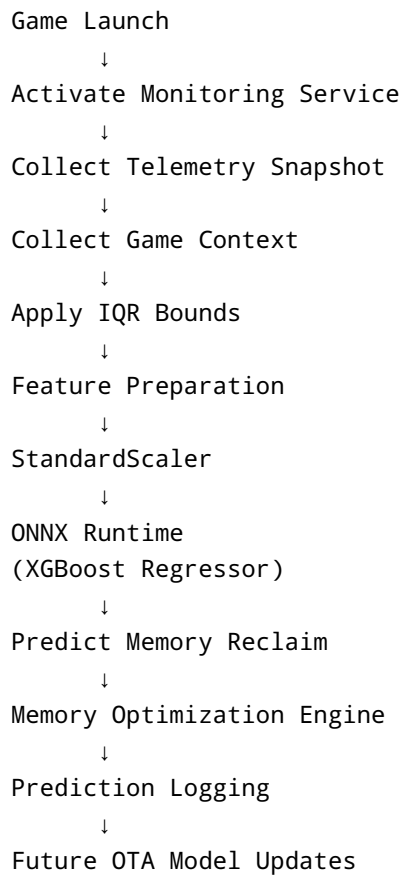
Total Runtime	~8 ms

Resource usage:

CPU Overhead	< 0.03%
Memory Usage	< 5 MB
Battery Impact	Negligible

The deployment therefore remains effectively invisible to the user while continuously monitoring gaming memory behavior.

End-to-End Deployment Flow



```
graph TD; A[Game Launch] --> B[Activate Monitoring Service]; B --> C[Collect Telemetry Snapshot]; C --> D[Collect Game Context]; D --> E[Apply IQR Bounds]; E --> F[Feature Preparation]; F --> G[StandardScaler]; G --> H["ONNX Runtime  
(XGBoost Regressor)"]; H --> I[Predict Memory Reclaim]; I --> J[Memory Optimization Engine]; J --> K[Prediction Logging]; K --> L[Future OTA Model Updates];
```

Game Launch

↓

Activate Monitoring Service

↓

Collect Telemetry Snapshot

↓

Collect Game Context

↓

Apply IQR Bounds

↓

Feature Preparation

↓

StandardScaler

↓

ONNX Runtime
(XGBoost Regressor)

↓

Predict Memory Reclaim

↓

Memory Optimization Engine

↓

Prediction Logging

↓

Future OTA Model Updates